

e-Borrow

Andrea Mirarchi

1 Cosa fa il sistema

e-Borrowing è una web application ispirata a un marketplace come eBay, ma con un obiettivo diverso: lo scambio di oggetti in prestito invece della vendita.

Gli utenti possono:

- pubblicare richieste di oggetti in prestito
- pubblicare offerte di prestito
- prendere in carico richieste esistenti

Il sistema include una gestione automatica della priorità delle richieste e un meccanismo di ordinamento delle offerte.

2 Casi d'uso

2.1 Caso d'uso 1 — Creazione richiesta

Un utente:

- inserisce una richiesta di prestito (oggetto, messaggio, tempo limite)
- la richiesta viene pubblicata e resa visibile agli altri utenti
- altri utenti possono consultarla e prenderla in carico

2.2 Caso d'uso 2 — Creazione offerta

Un utente:

- pubblica un oggetto disponibile al prestito
- inserisce le informazioni relative all'offerta
- l'offerta viene resa pubblica

Le offerte vengono mostrate in ordine di rilevanza rispetto alla domanda degli utenti.

2.3 Caso d'uso 3 — Presa in carico

Un utente:

- seleziona una richiesta
- decide di prenderla in carico
- la richiesta viene associata a un prestatore
- il prestito viene formalizzato nel sistema

3 Come è implementato

3.1 Architettura backend

Il backend è sviluppato in Flask con architettura a livelli:

- app.py: routing e gestione HTTP
- services: business logic
- models: entità del dominio
- utils: algoritmi (AI, ranking, matching)
- repository: persistenza dati

3.2 Matching e ordinamento

Il sistema di associazione tra richieste e offerte è gestito tramite l'utility **matching**, che confronta gli oggetti in base al nome e determina le corrispondenze tra domanda e offerta.

3.3 Priorità delle richieste

La priorità delle richieste viene calcolata con la formula:

$$\text{Priorità} = \text{Urgenza} + \text{Importanza} - \text{Tempo}$$

Urgenza e importanza vengono stimate automaticamente attraverso un sistema di analisi del contenuto testuale basato su un dataset di keyword e un modello AI leggero per lo scoring semantico del messaggio.

3.4 Gestione del prestito (Loan)

Quando una richiesta viene presa in carico, il sistema crea un'entità **Loan** che rappresenta il prestito attivo.

Il Loan:

- collega richiedente e prestatore
- rappresenta lo stato attivo del prestito
- viene generato dal service layer al momento della presa in carico

3.5 Frontend

Il frontend utilizza:

- Jinja2 come template engine
- web components
- Bootstrap per lo stile

3.6 Modello logico

Il sistema è basato su ereditarietà:

- BorrowEntity come classe base
- BorrowRequest per le richieste degli utenti
- Offer per le offerte di prestito
- Loan per i prestiti attivi

3.7 Persistenza dati

Il database è implementato tramite file JSON Lines (.jsonl), utilizzato perché consente scritture append-only più sicure e semplici rispetto a un singolo file JSON.

3.8 Validazione dati

Il sistema utilizza due livelli di validazione:

- WTForms per la validazione dei form HTML
- Marshmallow per la validazione delle API JSON

3.9 Routing

La gestione delle route e delle API è basata su Flask tramite il decoratore `@app.route`.

3.10 Configurazione

Il progetto utilizza:

- python-dotenv
- file .env per variabili d'ambiente
- configurazioni centralizzate per sicurezza e flessibilità

3.11 Business logic

La logica principale del sistema è centralizzata nel service layer, che:

- crea richieste e offerte
- calcola le priorità
- gestisce il matching tra utenti
- genera i prestiti (Loan)